23CSE373 · COMPUTER VISION · GROUP 10

Fast Object Proposal Generation Using Handcrafted Gradient Features

A from-scratch re-implementation and study of BING — Binarized Normed Gradients for Objectness Estimation (Cheng, Zhang, Lin, Torr · CVPR 2014)

Abhinav Dileep AM.SC.U4CSE23202

Ayyappadas M T AM.SC.U4CSE23209

Hari Sankar A AM.SC.U4CSE23221

Parthiv M AM.SC.U4CSE23240

Sai Kishen K M AM.SC.U4CSE23248



Why object proposals?

- **Object detection is expensive.** A sliding-window detector must classify every position × scale × aspect ratio, which is 10^5–10^6 windows per image.
- **Most windows are background.** Only a handful of windows actually contain an object.
- **Object proposals.** A cheap, class-agnostic objectness score ranks the windows, so the detector only examines the top ~1000.
- **Base paper: BING (CVPR 2014).** Handcrafted gradient features + bit operations produce proposals at 300 fps, with no deep learning.

$\sim 10^5+$

windows a naïve sliding-window scan evaluates per image

~1000

proposals that a detector needs to check instead

300 fps

BING's reported speed in the paper's C++ implementation

Problem definition

Given an image I, output a ranked list of N bounding boxes such that as many objects as possible are covered (IoU ≥ 0.5) by a small N, in milliseconds, using only handcrafted features.

1

Input

An RGB image of any size, any content

2

Output

Boxes (x_1, y_1, x_2, y_2) ranked by objectness score, plus an objectness heat-map

3

Constraints

Class-agnostic, real-time, CPU only, no deep networks

4

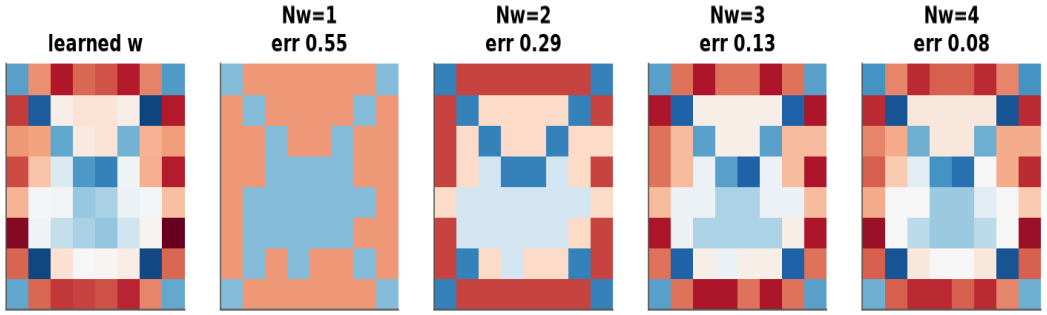
Success measures

Detection rate vs #proposals (DR@N), MABO (box tightness), runtime (fps)

Scope: we answer “Which regions are likely to contain objects?”, not “What is the object?” (classification is a later stage).

Proposed solution: BING, rebuilt and extended

- **Key idea.** Objects have closed boundaries. Shrink any window to 8×8 and look at gradient magnitudes: object windows show a ring of edges.
- **64-weight linear model.** An SVM filter learns this “closed boundary” template (right); a second stage calibrates each window size.
- **Bits instead of floats.** Filter and features are binarized, so each window costs a few AND + POPCOUNT operations.



Our novelty beyond the paper

Stage III re-ranking

Colour, edge and geometry cues re-order BING's windows

Complete detector

HOG or CNN features + linear SVMs on our proposals, scored by VOC mAP

Head-to-head on VOC

Authors' C++ code, same images and evaluator; two missing details found

From-scratch Python

Numba kernels of the paper's Alg. 2, now multi-threaded

System design: 8-stage pipeline

1

Input image

longer side ≤ 500 px

→

2

Pre-processing

colour space: RGB / HSV / Gray / Lab

→

3

Candidates

36 power-of-two sizes (16… 512, clipped), every 8×8 position

→

4

Gradient features

image resized per size, [-1 0 1] gradient

5

Binarized NG

top Ng=4 bit-planes packed into 64-bit words

→

6

Objectness score

s = ⟨w, g⟩ via AND + POPCOUNT (Nw=2)

→

7

Ranking

greedy NMS per score map, per-size calibration v·s + t

→

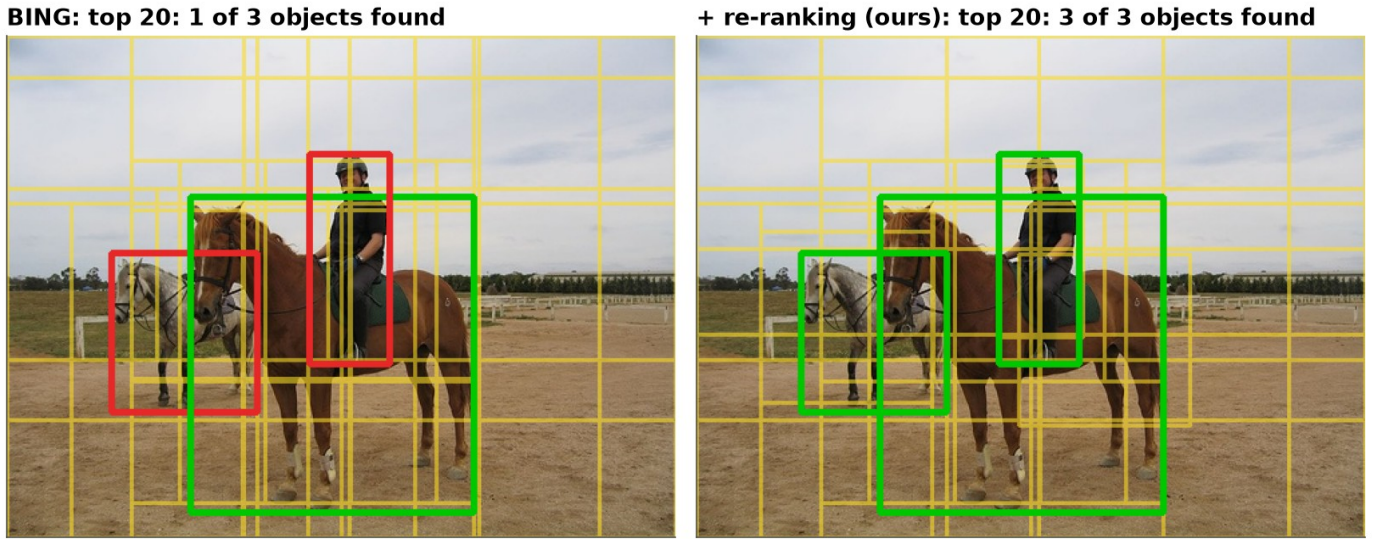
8

Top proposals

sorted boxes + objectness heat-map

Training: stage I is a linear SVM on 8×8 NG features of object boxes vs random background windows; stage II is a 1-D calibration per window size. Our stage III then re-ranks the output (next slide).

Our novelty: multi-cue re-ranking (stage III)



Top 20 windows (yellow) on a VOC 2007 test image. Green: object found (IoU ≥ 0.5); red: missed.

- **Problem.** BING ranks windows with a single cue, its 8×8 edge template, so object windows can sit far down the list.
- **Our idea.** Re-score BING's windows with extra cheap, handcrafted cues, learned from VOC trainval.

1 · Colour contrast
Lab colour inside the window vs. a ring around it

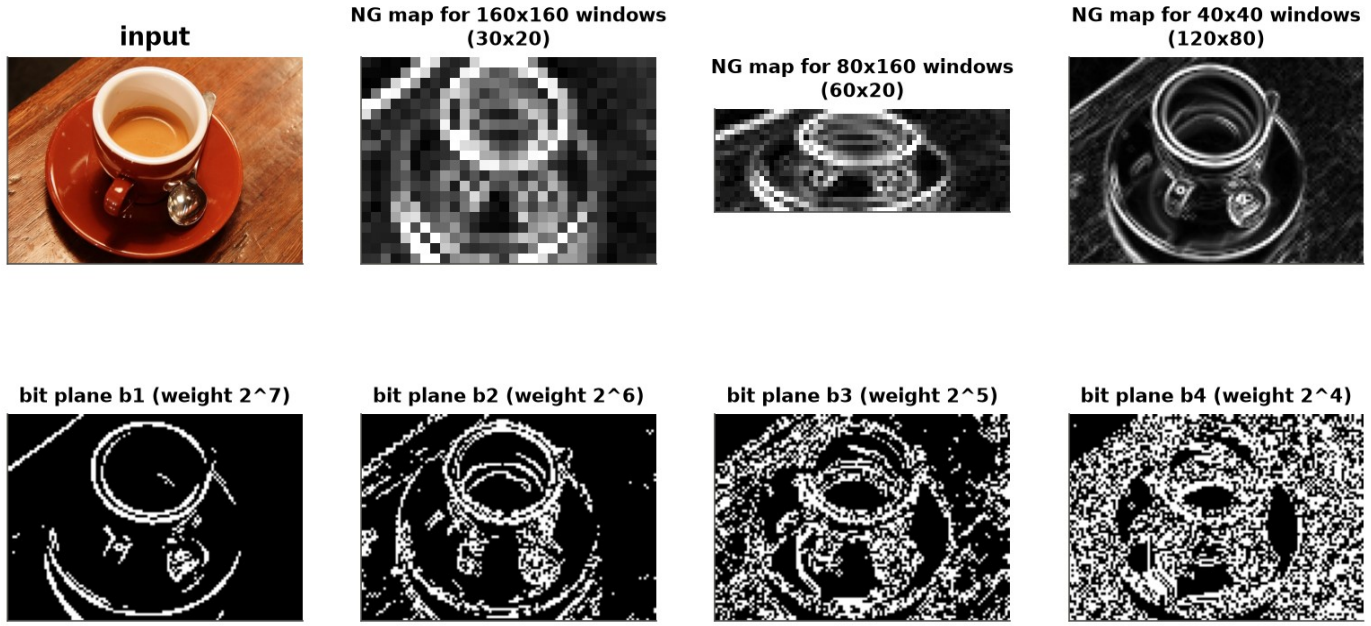
2 · Edge density
Gradient energy inside vs. the ring

3 · Geometry
Position, size and aspect ratio of the window

4 · BING itself
Its calibrated score and rank

A small gradient-boosted tree model combines the 16 values into a new score. Every cue is read from integral images in O(1): no deep network.

Normed gradients → bit operations



Top: NG maps for three window sizes (each 8×8 patch = one window). Bottom: the 4 bit-planes forming the binarized feature.

- Implemented in Python: bing/ package (features, binary, Numba fast path, model, metrics, datasets); unit tests prove bitwise score = float score.

Approximate the filter

$w \approx \Sigma_j \beta_j a_j, \quad a_j \in \{-1, +1\}^{64}$

Approximate the feature

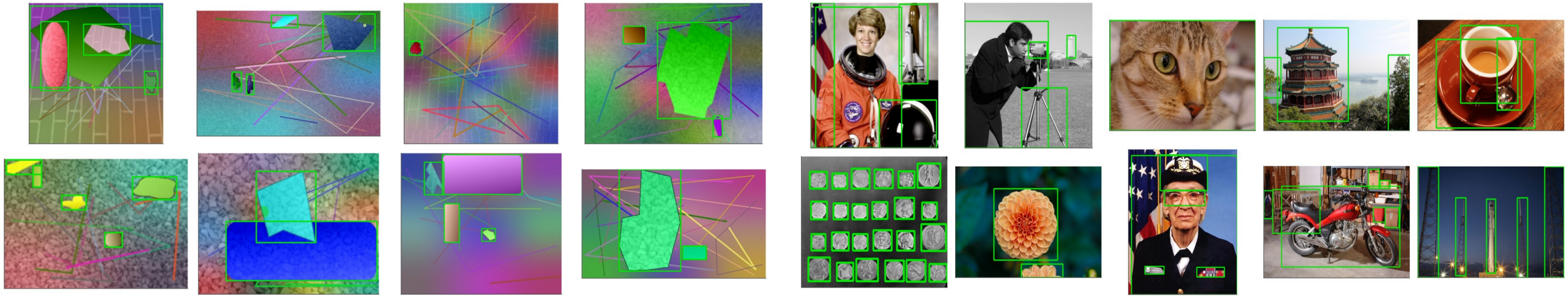
$g \approx \Sigma_k 2^{(8-k)} b_k, \quad b_k \in \{0, 1\}^{64}$

Score with bits only

$\langle a_j, b_k \rangle = 2 \cdot \text{popcnt}(a_j^+ \text{ AND } b_k) - \text{popcnt}(b_k)$

12 POPCOUNTs per window instead of 64 multiply-adds.

Experimental setup



PASCAL VOC 2007

The paper's benchmark: train on 5011 trainval images, test on 4952 test images (12,032 objects, “difficult” ones ignored).

Synthetic scenes

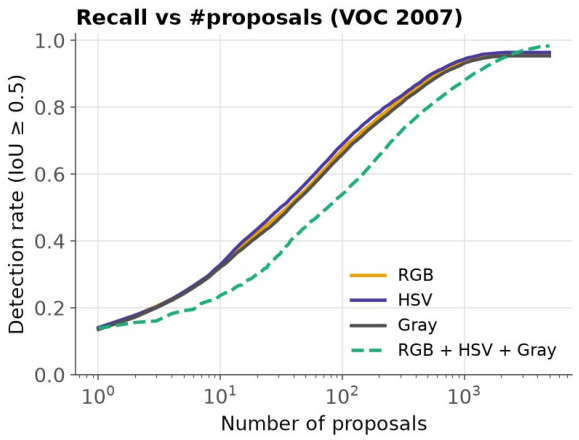
600 train / 300 test images: shapes with texture and shading on cluttered backgrounds (edges that are not objects).

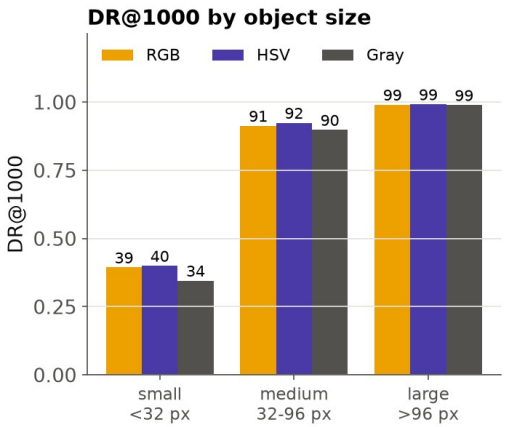
Real photographs

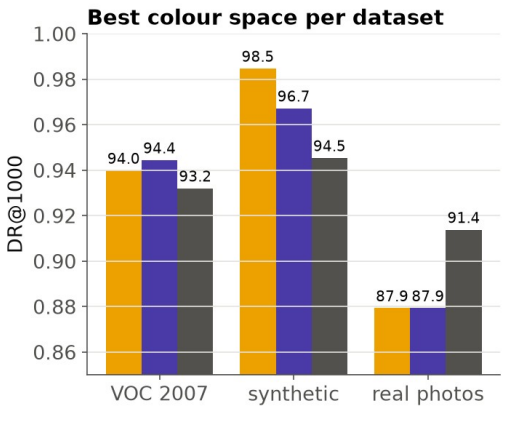
10 photos, 58 annotated objects, never used for training: tests generalisation.

Metrics: detection rate DR@N (share of objects covered with IoU ≥ 0.5 by the top N proposals), MABO (mean best overlap), images/s; for the detector, VOC mean average precision (mAP). Baselines: the authors' own C++ BING, Selective Search, sliding windows, random boxes. All scored with the same evaluator.

Design choice: RGB, HSV or Gray?







- **Setup:** one BING model per colour space, trained on VOC trainval; the gradient is the strongest of the channels.
- **VOC:** HSV is best and wins 15 of 20 classes at 100 boxes. Gray misses colour-only edges, most on small objects (34.5% vs 39.3%).
- **Other data:** RGB is best on the synthetic scenes; Gray transfers best from synthetic training to real photos.
- **Merging all three** (the paper's BING-Diversified) only pays off at large budgets.

94.4%

HSV: VOC objects in the top 1000 (RGB 94.0%, Gray 93.2%)

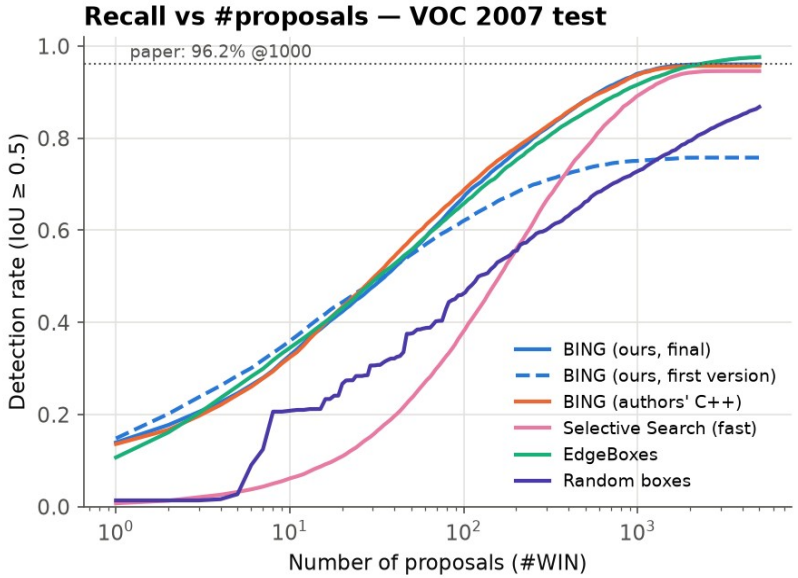
98.4%

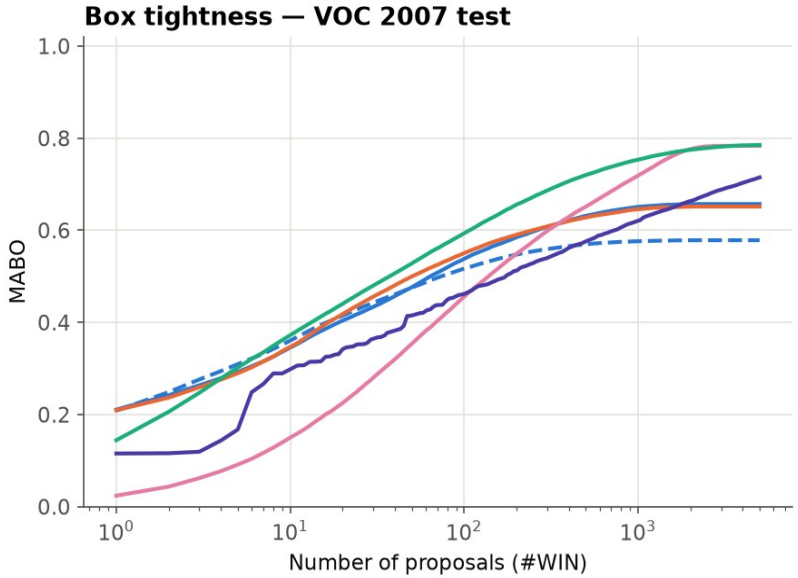
all three merged, top 5000 (best single: 96.4%)

91.4%

Gray on real photos, trained on synthetic only (RGB, HSV: 87.9%)

PASCAL VOC 2007: reproducing the paper





94.0%

objects found in the top 1000, our final BING

93.8%

the authors' C++ code, same images and evaluator

96.2%

reported in the paper (authors' evaluator)

- **Our rebuild matches the authors' code** on their own benchmark: 94.0% vs 93.8% at 1000, 96.0% vs 95.7% at 5000; box tightness 0.651 vs 0.646.
- **By the authors' own evaluator** their code scores 96.0% here, so the paper's number reproduces.
- **Unseen classes (the paper's test):** trained on only 6 classes, BING finds 91.1% of objects of the other 14 (trained on all 20: 93.3%).

Comparison with existing methods

Method	VOC DR@1k	VOC MABO	Synth. DR@1k	Real DR@1k	img/s (1 thread)
BING — ours (Python + Numba)	**94.0%**	**0.651**	**98.5%**	**87.9%**	**98**
BING — authors' C++ code	93.8%	0.646	94.9%	75.9%	76
Selective Search (fast)	89.2%	0.719	97.5%	100.0%	1.3
EdgeBoxes	91.6%	0.753	—	—	6.5
Random boxes	72.9%	0.621	53.6%	48.3%	—

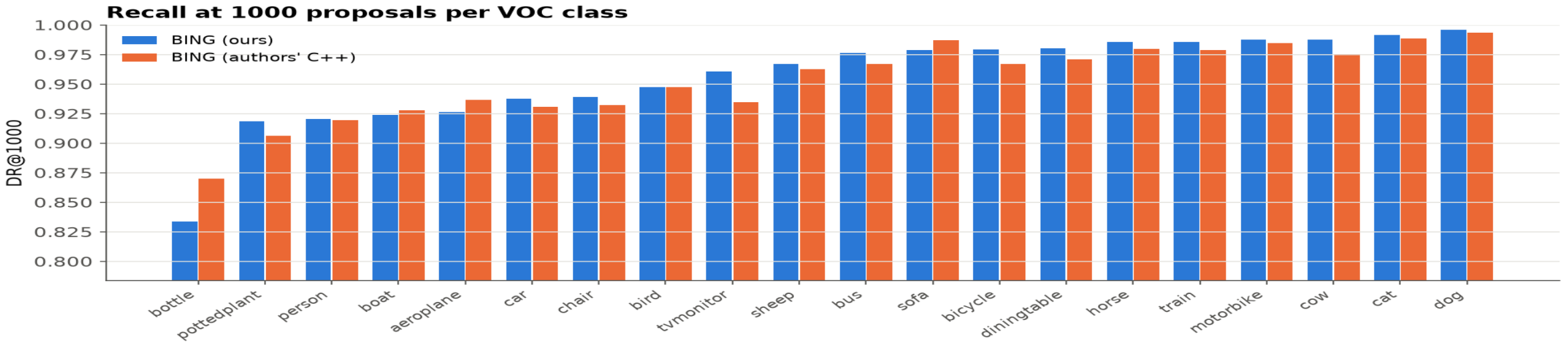
All methods on all 4,952 VOC test images; speed on one thread, all timed on the same machine. With 4 threads, ours reaches 294 img/s and the authors' code 300.

- **BING vs Selective Search and EdgeBoxes:** BING finds the most objects in the top 1000 (94.0% vs 89.2% and 91.6%) and is 15–77× faster, but its boxes are looser (MABO 0.65 vs 0.72 and 0.75).
- **Ours vs the authors' code:** same accuracy on VOC; ours generalises better to real photos when trained on synthetic data, and is faster on one thread (98 vs 76 img/s).

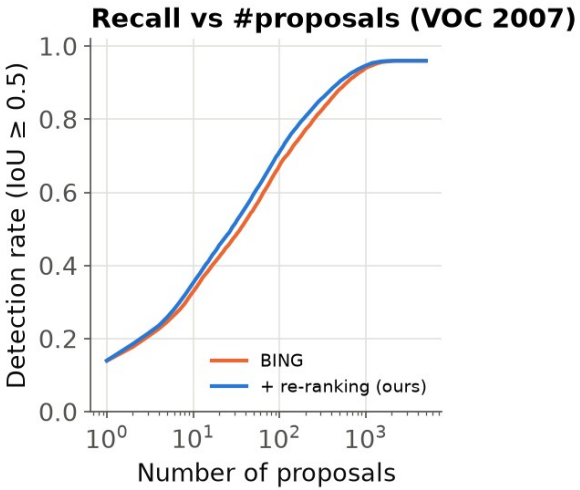
Two details the paper leaves out

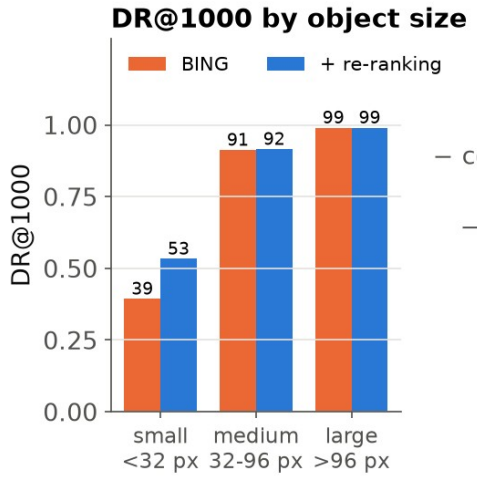
Our BING on VOC 2007	DR@1k	DR@5k	MABO
Paper's text: sizes 10–320, local-max NMS (first version)	75.1%	75.8%	0.576
+ released code's window sizes (16–512, clipped)	79.2%	79.7%	0.607
+ released code's greedy NMS only	92.2%	94.4%	0.621
Both (final)	**94.0%**	**96.0%**	**0.651**

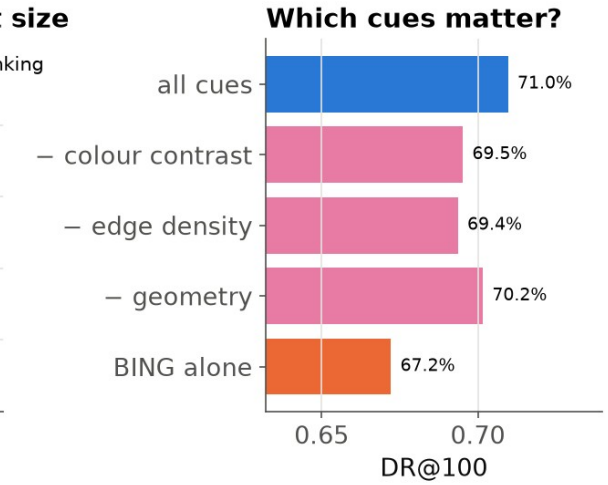
- **Window sizes:** the paper lists 10…320; the released code uses 16…512 and clips windows to the image, so near full-image boxes exist. VOC is full of large objects.
- **Suppression:** keeping only local maxima of each score map throws away good windows; the code's greedy suppression keeps the best 130 spread-out windows per size.
- **Lesson:** reproducing a paper needs its code, not just its text.



Novelty result: more objects, found sooner







- **Edge density is the key cue for small objects:** without it, small-object recall falls from 53.3% to 45.0%.
- **Cost:** 17 instead of 10 ms per image on one thread, still 46× faster than Selective Search.
- **Tried and rejected:** snapping box edges to strong gradients and adding finer window sizes; neither raised recall on the full test set.

71.0%

objects found in the top 100 (BING alone: 67.2%)

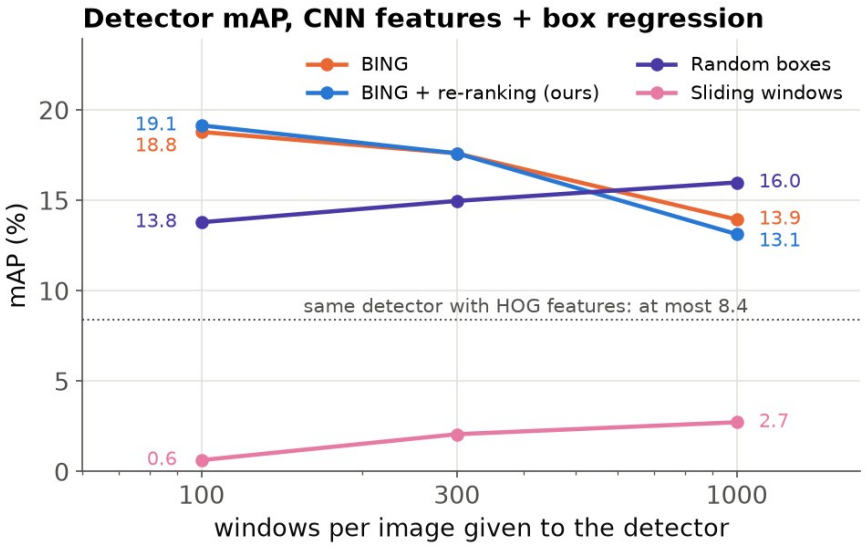
53.3%

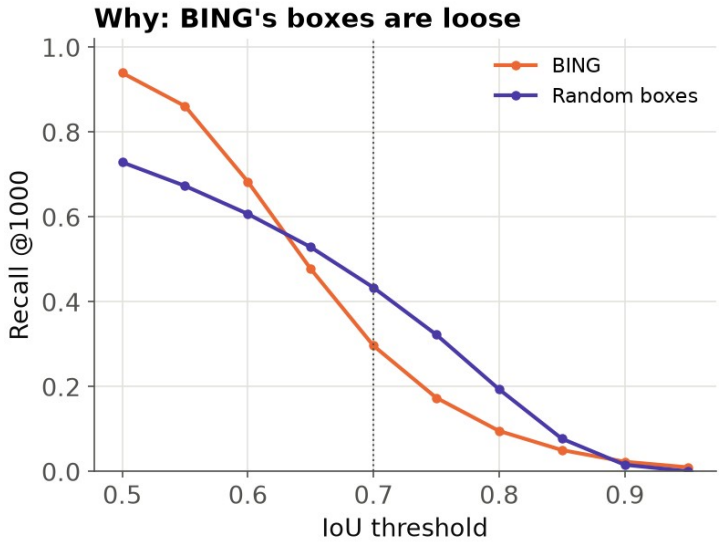
small objects found in the top 1000 (BING alone: 39.3%)

94.7%

objects found in the top 1000 (BING alone: 94.0%)

A complete detector built on our proposals





19.1%

mAP, CNN detector on our re-ranked top 100 (HOG: 7.4%)

0.6%

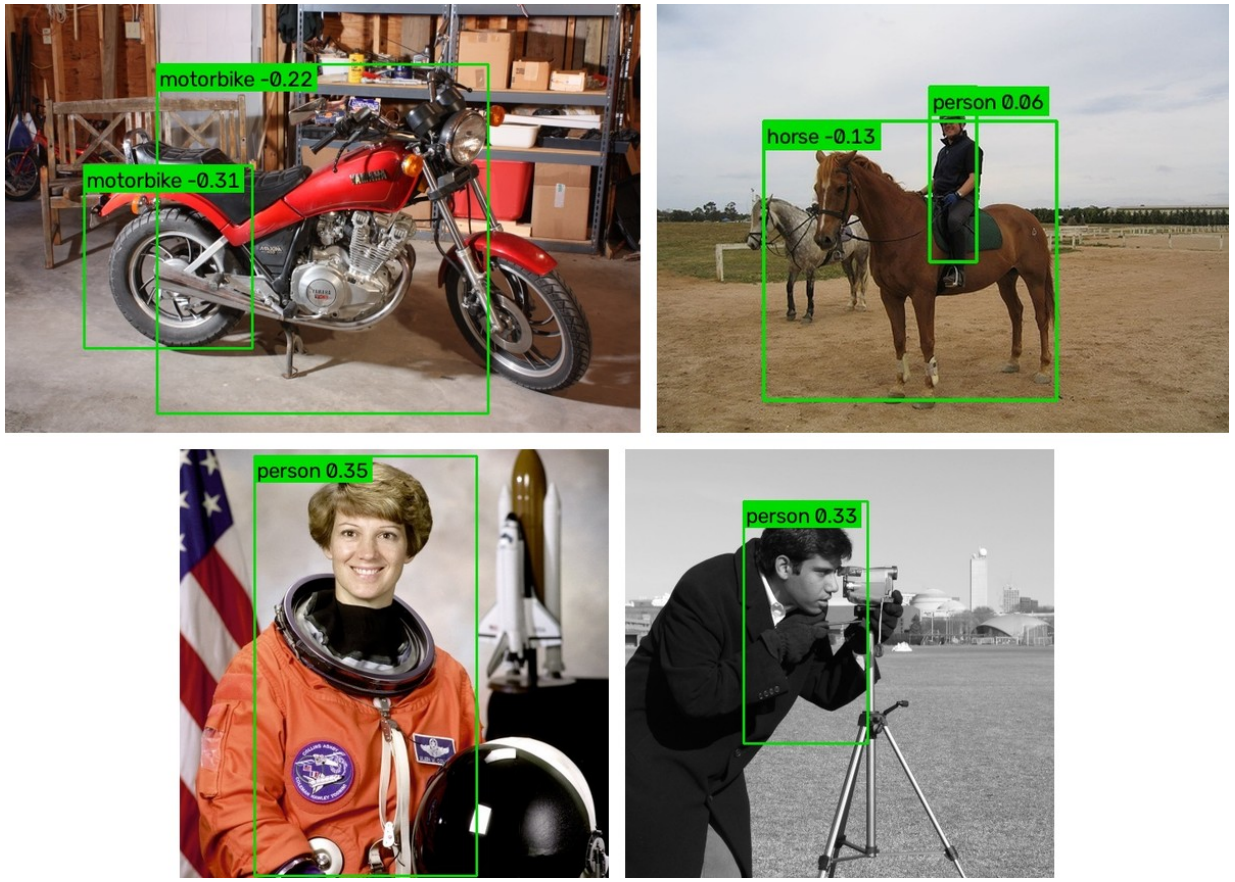
mAP from 100 sliding windows, same detector (30× lower)

- **Pipeline:** each window → mean ResNet-50 conv4 + conv5 activations (ImageNet, frozen) → 20 linear SVMs + box regression. A fully handcrafted version (HOG + colour) reaches 7.4%.
- **Finding:** at 100 windows BING beats random boxes (18.8% vs 13.8%); at 1000, random boxes win (16.0%) because BING's boxes are loose. Hosang et al. (TPAMI 2016): recall at stricter overlaps predicts detection quality.

30%

of objects covered at IoU ≥ 0.7 by BING's top 1000 (random boxes: 43%)

The detector in action



Our CNN detector on the top 100 re-ranked BING boxes. Label = VOC class, number = classifier score.

What it finds
The motorbike, the brown horse and its rider, the astronaut and the cameraman, each named with one of the 20 VOC classes.

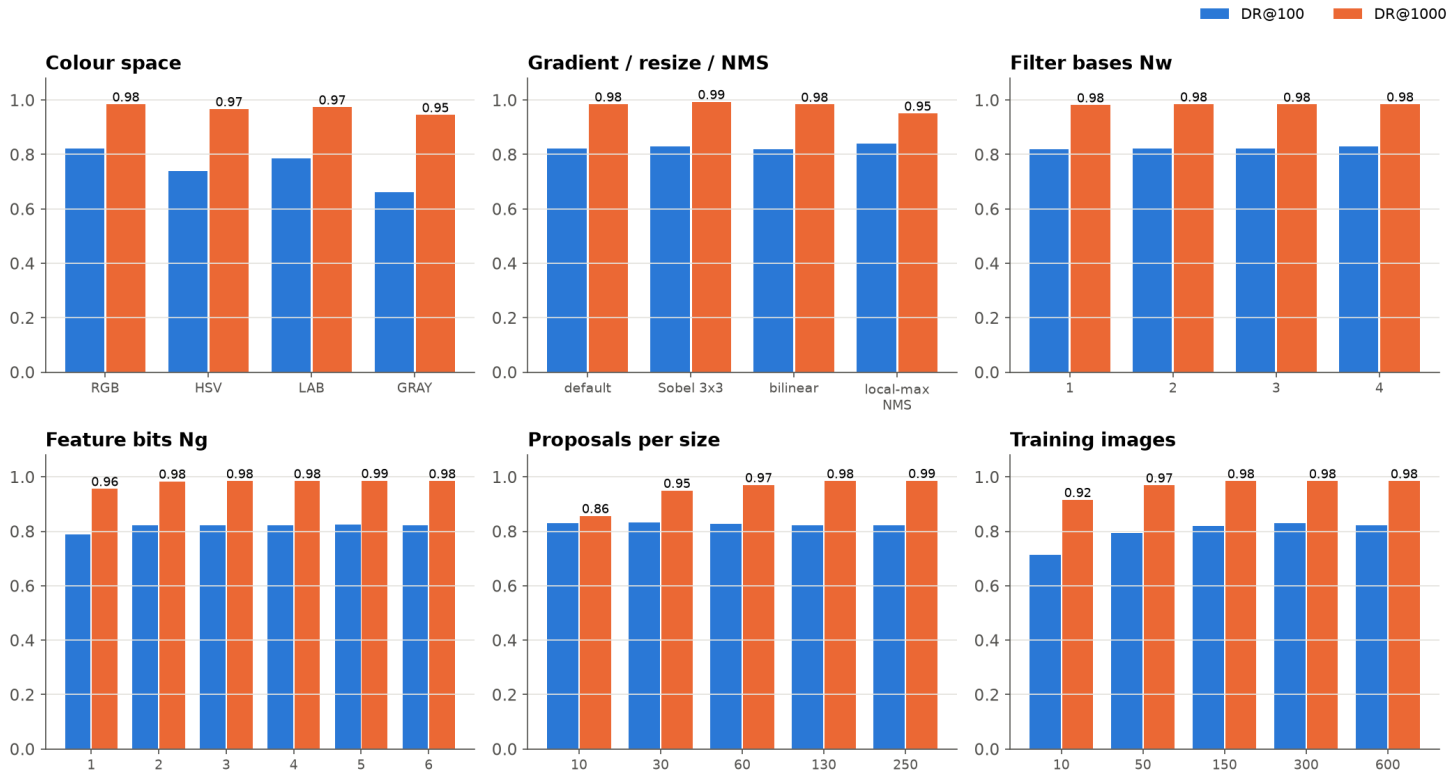
Limits
It misses some objects (the white horse). Only VOC classes are known: a cup or a phone gets no label. Boxes can be loose: the motorbike box cuts off the front wheel.

Try it

```
python scripts/demo.py
  --image photo.jpg --detect --show
```

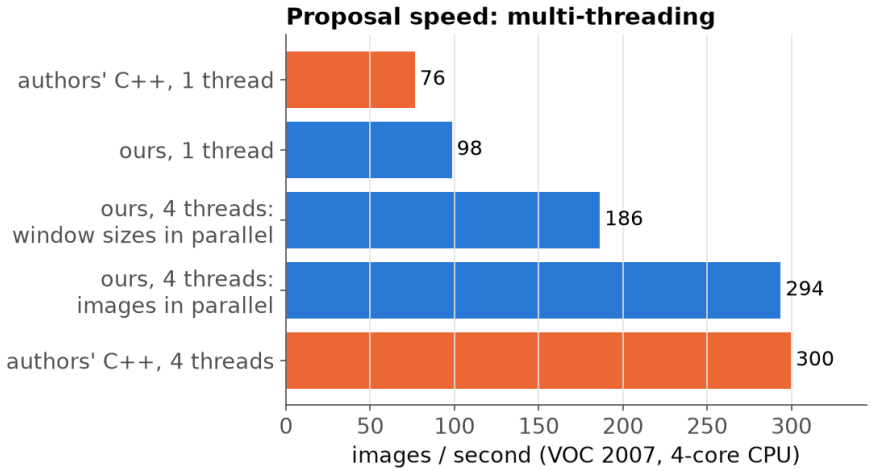
BING says where objects are; the detector says what they are.

What matters? Ablations and speed



DR@1000 on the synthetic test set (bars: blue DR@100, orange DR@1000).

Limitation: power-of-two sizes and an 8-px stride give coarse boxes (VOC MABO 0.65); small objects such as bottles are hardest.



- **Gradient:** [-1 0 1] 98.5% ≈ Sobel 99.3%
- **Colour:** RGB 98.5% > Lab 97.4% > HSV 96.7% > Gray 94.5%
- **Bits:** Ng = 1 → 95.6%; Ng ≥ 2 → 98.4%
- **Data:** 10 images → 91.6%; 600 → 98.5%
- **Speed:** 4 threads, one image each → 294 img/s (authors' C++ on the same machine: 300)

CONCLUSION

Gradients, a 64-weight model and bit operations find 94% of VOC objects, as the authors' code does; our re-ranking finds them sooner.

94.0%
VOC 2007 objects found in top 1000 (authors' code 93.8%)

71.0%
found in the top 100 with our re-ranking (BING: 67.2%)

53.3%
small objects in the top 1000 with re-ranking (BING: 39.3%)

294
images / second, 4 threads (authors' C++: 300)

Future work tighter boxes (better box regression, finer sizes) · fine-tuned CNN features · train-on-some-classes generalisation

References [1] M.-M. Cheng, Z. Zhang, W.-Y. Lin, P. H. S. Torr, “BING: Binarized Normed Gradients for Objectness Estimation at 300fps,” CVPR 2014. [2] J. Uijlings et al., “Selective Search for Object Recognition,” IJCV 2013. [3] B. Alexe, T. Deselaers, V. Ferrari, “Measuring the Objectness of Image Windows,” TPAMI 2012. [4] J. Hosang et al., “What makes for effective detection proposals?,” TPAMI 2016.

Thank you